

Designing a Rigorous, Reproducible Benchmark Study of GPU-Accelerated ML Inference on Local Kubernetes

Executive summary

A strong “laptop GPU + Kubernetes” inference benchmarking paper is won or lost on experimental control: pinning the GPU/driver/container/Kubernetes stack, separating *engine-level* performance from *platform/operator* overhead, and reporting full latency distributions (especially tail latency) under realistic traffic patterns rather than a single “throughput” number. [\[filecite turn0file0\]](#)

To produce a paper that is both publishable and reproducible on commodity hardware, the highest-value research focus areas are:

- **Throughput-latency tradeoffs under batching and queueing** (e.g., concurrency sweeps and rate-controlled arrival) with explicit SLO targets; this is a central theme in modern LLM serving research. [1](#)
- **VRAM-pressure and memory management envelopes** (how far you can push context length / batch size / concurrency before instability or OOM), which is particularly constraining on laptop GPUs. [2](#)
- **Kubernetes-layer effects** (container runtime + device plugin/operator + scheduling semantics) and how they perturb repeatability on a single node. [3](#)
- **Operability tradeoffs**: what additional control/observability you get from each serving stack (metrics endpoints, standardized protocols, autoscaling knobs) versus added overhead and complexity. [4](#)

A practical, defensible “core comparison set” for experiments on a single laptop GPU is: (a) a direct model server, (b) a multi-framework inference server with mature metrics tooling, and (c) an orchestration layer that still exercises Kubernetes-centric deployment. Your survey can be broader, but your deepest experiments should stay narrow to keep results interpretable and to avoid “shallow comparisons across too many systems.” [5](#)

Research questions and hypotheses

The goal is to define questions that are answerable *on a single GPU*, yield publishable insight, and are testable with repeatable sweeps.

Research question about layering and fairness

How much of observed performance difference comes from the *execution engine* versus the *serving platform layer* (routing, adapters, proxies, operator sidecars)? Hypothesis: platform layers add measurable overhead

(especially for short requests), but may reduce run-to-run variance and improve observability and deployment ergonomics. ⁶

Research question about throughput-tail-latency curves

For fixed workload shapes (input size, output size, batch/concurrency), what are the Pareto frontiers of throughput vs p99 latency across systems? Hypothesis: systems that emphasize scheduler design and continuous batching will exhibit higher throughput at a fixed tail-latency SLO, but may show earlier “knee points” under bursty traffic due to queue buildup. ⁷

Research question about KV-cache and VRAM envelopes (LLMs)

Under increasing context length and concurrency, where do OOM/instability boundaries occur, and how does memory-management strategy influence them? Hypothesis: paged KV-cache approaches reduce fragmentation and extend stable operating regions, especially for longer sequences. ⁸

Research question about batching knobs and queue delay

When you vary batching/queue-delay controls, how do you trade mean latency vs tail latency vs throughput? Hypothesis: adding modest queue delay improves throughput (more batching opportunities) while disproportionately harming tail latency beyond a threshold; the “best” value depends strongly on arrival burstiness. ⁹

Research question about protocol standardization

Does adopting a standardized inference protocol reduce client/tooling complexity and improve comparability across serving stacks? Hypothesis: using the Open Inference Protocol (V2) enables more consistent workload replay and allows the same client harness to target multiple inference servers, reducing confounds introduced by per-system request formats. ¹⁰

Research question about autoscaling on a single-GPU node

Do Kubernetes autoscaling mechanisms help, harm, or become irrelevant on a single GPU? Hypothesis: in a single-node, single-GPU setting, replica scaling cannot increase GPU capacity unless you explicitly enable GPU sharing; autoscaling may still matter for CPU-bound preprocessing or for scale-to-zero/idling behavior. ¹¹

Inventory checklist for a laptop GPU and local Kubernetes

This section is designed to be copy-pasted into your paper’s “Experimental Setup” and your artifact repository’s README. Items marked “Fill in” are the unknowns you said to assume.

Fill-in-the-blanks system description

Category	What to record (paper + repo)	How to obtain	Why it matters
Laptop GPU	Fill in: GPU model, VRAM size, PCIe info, max power	<code>nvidia-smi -q</code> (host) ¹²	Determines feasible models, KV-cache size, and power/thermal behavior

Category	What to record (paper + repo)	How to obtain	Why it matters
GPU driver	Fill in: driver version	<code>nvidia-smi</code> or <code>nvidia-smi --query-gpu=driver_version</code> ¹²	Container compatibility and reproducibility
Host OS	Fill in: distro + kernel + power mode	<code>uname -a</code> , <code>/etc/os-release</code> (methodological)	Laptop throttling and scheduler differences can shift tail latency
Container runtime	Fill in: Docker / containerd / CRI-O + versions	<code>docker version</code> / <code>containerd --version</code>	GPU runtime configuration differs across runtimes ¹³
GPU container enablement	NVIDIA Container Toolkit version + config method	<code>nvidia-ctk --version</code> ; runtime configure steps ¹⁴	Required for GPU pass-through into containers
Kubernetes distribution	Choose one: minikube ¹⁵ / MicroK8s ¹⁶ / k3s ¹⁷ ; Fill in: version	<code>kubectl version</code> ; distro-specific commands	GPU setup paths differ; consistency reduces confounds ¹⁸
GPU advertisement to K8s	Device plugin vs operator; versions	<code>kubectl describe node</code> for <code>nvidia.com/gpu</code>	Required for scheduling; impacts sharing options ¹⁹
Monitoring stack	Prometheus/metrics stack + versions	Helm release manifests (methodological)	Ensures metrics are collected identically across runs

Critical GPU-on-Kubernetes checklist (drivers → runtime → scheduling)

Host validation (must pass before any benchmarking)

- 1) Confirm the host sees the GPU and reports power/temperature/utilization fields: `nvidia-smi`. ¹²
- 2) Confirm you can log power/thermal/utilization time series for later correlation with latency spikes. NVIDIA documents query patterns and power reporting via NVML. ²⁰

Container runtime configuration (must be pinned and scripted)

NVIDIA's official guidance is to configure runtimes using `nvidia-ctk runtime configure` (Docker, containerd, CRI-O). ¹⁴

Example (containerd on Linux, typical for local Kubernetes distributions):

```
sudo nvidia-ctk runtime configure --runtime=containerd
sudo systemctl restart containerd
```

13

Example (Docker engine, typical for a Docker-driver local cluster):

```
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

21

Kubernetes GPU scheduling semantics (avoid subtle misconfigurations)

Kubernetes treats GPUs as extended resources; you may set GPU `limits` without `requests` (limit becomes request), but if you set both they must match, and you cannot set a GPU request without a GPU limit. ²²

Minimal GPU test pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-smoke
spec:
  restartPolicy: Never
  containers:
  - name: cuda
    image: nvidia/cuda:12.3.2-base-ubuntu22.04
    command: ["bash", "-lc", "nvidia-smi && sleep 5"]
    resources:
      limits:
        nvidia.com/gpu: 1
```

(Interpretation: if this fails, *do not* proceed to inference benchmarking.)

Local Kubernetes GPU bring-up recipes (pick one and stick to it)

1) **minikube**: The GPU tutorial explicitly instructs installing the NVIDIA Container Toolkit, configuring Docker with `nvidia-ctk`, and (for GPU usage) using its GPU options. ²¹

2) **MicroK8s**: The GPU add-on is documented as enabling NVIDIA GPU support using the GPU Operator, configuring `nvidia-container-runtime` for containerd, and setting up the `nvidia.com/gpu` device plugin. ²³

3) **k3s**: Documentation notes that if you have not changed the default runtime on GPU nodes, you must explicitly request the NVIDIA runtime via `runtimeClassName: nvidia`. ²⁴

Survey of Kubernetes inference frameworks and operators

A key framing step for a rigorous paper is to classify systems by their *layer*: GPU enablement, model server, and orchestration/operator. Comparing systems across different layers without controlling the underlying runtime creates invalid conclusions. `filecite{turn0file0}`

Comparative landscape table

Component (layer)	What it is (for benchmarking)	Strengths for a paper	Weaknesses / confounds	Maturity signals	Install/config notes for a single laptop GPU
KServe (operator)	Operator with CRDs like <code>InferenceService</code> , plus standardized data-plane concepts and optional modes. ²⁵	Lets you test “operator overhead + standardized protocols + autoscaling modes” using a consistent CRD-driven deployment workflow. ²⁶	Heavy dependency surface if you choose serverless mode; more moving parts than direct Deployment-based serving. ²⁷	CNCF incubating as of 2025 (signal of ecosystem maturity). ²⁸	Choose <i>Standard</i> mode for minimal deps (Deployments + HPA). <i>Knative</i> mode adds request-based autoscaling and scale-to-zero. ²⁹
KFServing (historical name)	Rebranded project lineage (important for literature review and older configs). ³⁰	Helps interpret older papers/scripts and avoid “naming drift” reproducibility issues. ³⁰	Mixed API groups and manifests across eras can silently break installs. ³¹	Transition was formally announced; references remain in older repos. ³²	In paper artifacts, document mapping of old API groups to new ones if you reuse legacy YAML. ³¹
ModelMesh (KServe option)	Multi-model serving management/routing layer; designed for high-density and frequent model churn. ³³	Enables experiments on model cache/multi-model loading policies (if your paper includes multi-model scenarios). ³⁴	Adds complexity and extra control-plane interactions; may be overkill for single-model LLM tests. ³⁴	Described by maintainers as “mature” and aimed at high-density use cases. ³⁵	For a laptop paper, use sparingly: it is valuable for a multi-model appendix rather than the main comparison. ³⁴

Component (layer)	What it is (for benchmarking)	Strengths for a paper	Weaknesses / confounds	Maturity signals	Install/config notes for a single laptop GPU
Kubeflow components (serving)	Documentation and UI integrations around KServe (e.g., Models Web App managing <code>InferenceService</code>). ³⁶	Good for an “MLOps usability” angle: lifecycle management, CRD status tracking, and reproducible YAML workflows. ³⁷	Adds substantial platform surface unrelated to single-node inference performance. ³⁸	Active documentation and integration hooks. ³⁹	If included, treat it as an operational/control-plane comparison, not an engine performance comparison. ³⁷
Seldon Core 2 (operator/platform)	Framework for deploying and routing modular AI apps; architecture includes a data-plane ingress and routes to model servers (e.g., MLServer/Triton). ⁴⁰	Strong for studying “network hops/proxies and standard protocol routing overhead,” plus pipeline or async patterns. ⁴¹	More “platform” than “single server”; Kafka can be a prerequisite for some pipeline features, adding background load/confounds. ⁴²	Actively documented; positions itself as MLOps/LLMOps framework on Kubernetes. ⁴³	If you benchmark it, control the number of proxies/hops and keep the underlying model server identical to isolate platform overhead. ⁴⁴
BentoML + Yatai (packaging + operator)	Yatai provides K8s-native workflow with a <code>BentoDeployment</code> CRD managed by an operator. ⁴⁵	Useful “developer workflow” baseline: standardized packaging, CRD-driven deployment, and GPU allocation patterns. ⁴⁶	Inference performance depends heavily on your service implementation; comparison can become “your handler code” rather than platform.	Active repos + explicit CRD/operator docs. ⁴⁷	Treat it as a “custom service framework” and lock down handler code, preprocessing, and batching settings for fairness. ⁴⁸

Component (layer)	What it is (for benchmarking)	Strengths for a paper	Weaknesses / confounds	Maturity signals	Install/config notes for a single laptop GPU
TorchServe (model server)	PyTorch-oriented model server.	Historically common baseline for PyTorch serving.	Official docs state “Limited Maintenance” with no planned updates/patches, which is a security and longevity concern for a “current systems” paper. ⁴⁹	Limited maintenance notice is explicit. ⁴⁹	If included, position as a “legacy baseline” and isolate it to non-LLM workloads (classification/embedding) to avoid unfair comparisons. ⁴⁹
Triton Inference Server (model server)	Multi-framework inference server with HTTP (8000), gRPC (8001), Prometheus metrics (8002), and standardized protocols based on KServe inference protocols with extensions. ⁵⁰	High-quality metrics story and mature benchmarking tooling (Perf Analyzer); strong for ONNX/TensorRT/PyTorch baselines and LLM backends. ⁵¹	Compatibility constraints: container tags map to CUDA driver requirements; if you don't pin carefully, results won't reproduce. ⁵²	Mature vendor-supported stack with public support matrices. ⁵³	For a laptop: choose a tag whose required driver matches your host driver using the official compatibility matrix, and record both. ⁵⁴
TensorRT-LLM backend (Triton LLM path)	Triton backend for serving TensorRT-LLM models; includes inflight batching and paged attention support in its implementation. ⁵⁵	Ideal for an “optimized engine vs PyTorch engine” comparison under identical prompt distributions and SLOs. ⁵⁶	Requires a build step for engines and careful config tuning (queue delay, max batch, etc.), which expands the experimental surface. ⁵⁷	Maintained as part of Triton ecosystem. ⁵⁸	Separate engine-build from inference runs (e.g., build job → immutable artifacts) to keep benchmarks comparable across repetitions. ⁵⁹

Component (layer)	What it is (for benchmarking)	Strengths for a paper	Weaknesses / confounds	Maturity signals	Install/config notes for a single laptop GPU
Ray Serve LLM + KubeRay (orchestrator/ operator)	Kubernetes-native deployment of Ray Serve apps via <code>RayService</code> CR; Ray Serve LLM aligns with vLLM configuration via <code>engine_kwargs</code> . ⁶⁰	Allows platform-level experiments: routing/ backpressure, serving graph overhead, and K8s-native management while keeping engine arguments comparable. ⁶¹	Adds distributed runtime overhead even on single node; careful CPU/ memory pinning needed to avoid confounding GPU tests. ⁶²	Actively documented; recommended production path is RayService on Kubernetes. ⁶³	Install operator via Helm and pin chart/app versions; use Ray's Prometheus/ Grafana guidance for metrics consistency. ⁶⁴
NVIDIA device plugin / GPU Operator (GPU enablement)	Device plugin is a DaemonSet that exposes GPUs and tracks health; GPU Operator can manage toolkit, device plugin, and telemetry (DCGM exporter) and supports GPU time-slicing. ⁶⁵	Required foundation for any Kubernetes GPU inference paper; time-slicing enables controlled GPU-sharing experiments. ⁶⁶	Time-slicing lacks memory/ fault isolation; results can become noisy due to interference. ⁶⁷	Official docs emphasize operator-managed stack and DCGM telemetry. ⁶⁸	Prefer “no sharing” baseline; add a second phase with time-slicing only if you can enforce tight experimental control. ⁶⁷

Recommended representative inference workloads and load profiles

A credible workload suite is small enough to run repeatedly on a laptop, but rich enough to expose bottlenecks (prefill vs decode, batching, preprocessing, protocol overhead).

Workload suite design principles

- Use **at least one non-LLM workload** (classification/embedding) and **at least one LLM workload** to avoid a paper that only generalizes to one model family. KServe explicitly positions itself as multi-framework across common ML model types. ⁶⁹
- For LLMs, prioritize **workload shapes** (input/output token lengths, concurrency, burstiness) over absolute model size; on a laptop, meaningful insight often comes from identifying the stable operating envelope and tail-latency behavior. ⁷⁰

- Keep request format consistent by using the **Open Inference Protocol (V2)** where applicable, especially when comparing multiple inference servers and clients. ⁷¹

Recommended representative workloads (examples)

Workload class	Primary goal	Model tier guidance (laptop-friendly)	Framework / format	Key knobs to sweep
Vision classification	Measure batching effects and kernel efficiency	“small→medium” CNN (batchable)	TensorFlow SavedModel or ONNXRuntime/ TensorRT via Triton-style serving patterns (format choice depends on your stack) ⁷²	Batch size (1–64), concurrency, request payload size
Embedding / encoder NLP	Stress sequence length sensitivity without generation	BERT-class encoder	PyTorch or ONNX (same tokenizer config across runs)	Sequence length (128/256/512), batch size, concurrency
LLM generation (interactive)	Platform overhead + TTFT + tail latency	0.5–1B model class; cap context to fit VRAM	OpenAI-compatible server paths and V2 protocol where applicable	short input (64–128 tokens), short output (32–64), concurrency sweep
LLM generation (long prefill)	KV-cache + prefill bottlenecks	same as above	same	long input (512–2048 tokens), moderate output (128–256)
LLM generation (decode-heavy)	Decode throughput limits	same as above	same	short input, long output (256+)

For LLM memory control, vLLM documents `--max-model-len` including an auto-fit option, and `--gpu-memory-utilization` as a direct VRAM budget knob; it also documents `--seed` for reproducibility. ⁷³ If you use vLLM through Triton’s vLLM backend, NVIDIA’s docs explicitly call out that vLLM defaults to consuming a large fraction of GPU memory and that you should tune `gpu_memory_utilization` in `model.json`. ⁷⁴

Quantization can be a second-phase dimension (“fit larger model under same VRAM”). vLLM documents quantization support including BitsAndBytes and AutoAWQ. ⁷⁵

If you include TensorRT-LLM, NVIDIA documents FP16/BF16 support and quantization recipes and matrices.

76

Load profiles and traffic patterns to test

Your load suite should include both **closed-loop** (concurrency-based) and **open-loop** (arrival-rate-based) patterns, because they reveal different failure modes (saturation vs queueing collapse).

Closed-loop saturation and concurrency sweeps

Use fixed concurrency (1, 2, 4, ...) and “send-next-when-done” loops to produce throughput-latency curves and identify knee points.

Open-loop arrival rate (SLO compliance under queueing)

Grafana k6 documents “constant-arrival-rate” as an open-model executor where iterations start independently of response time—this is well suited for evaluating when p99 latency violates an SLO under increasing offered load. 77

Burstiness and ramp patterns

vLLM’s benchmark CLI explicitly supports request-rate controls and a `--burstiness` knob (Poisson-like to burstier/uniform spacing) and a `--max-concurrency` cap. 78

Autoscaling triggers

- In KServe Standard mode, autoscaling uses Kubernetes HPA (Deployments/Services/Ingress/HPA); scale-to-zero is *not* supported in Standard mode per KServe docs. 79
- Knative Serving documents cluster-level enablement of scale-to-zero and its defaults; this matters if you test idle-to-busy transitions. 80
- If you need custom-metric scaling, the Kubernetes SIG Prometheus Adapter implements Kubernetes custom/external metrics APIs for HPA v2-style scaling. 81

On a single GPU, autoscaling experiments must explicitly state whether you: (a) only scale CPU-side components, (b) keep GPU-bound replica count fixed at 1, or (c) enable time-slicing (oversubscription), because those regimes measure different phenomena. GPU time-slicing is documented as multiplexing without memory/fault isolation. 66

Benchmarking tools, metrics, and how to collect them

A rigorous paper reports all results in terms of both **service-level** and **system-level** metrics, collected consistently across all systems.

Benchmarking tools

Server-provided or vendor-provided benchmark tools (preferred as “ground truth triangulation”)

- vLLM provides a benchmark CLI with explicit parameters for request rate, burstiness, and concurrency controls. 82
- Triton’s Perf Analyzer generates inference requests, measures throughput/latency over windows, repeats until values stabilize, and supports stabilizing by percentile. 83

General load generators (useful for cross-system uniformity)

- Locust provides an open-source, code-driven load testing approach for HTTP and other protocols, with real-time stats and export. ⁸⁴
- Vegeta emphasizes constant request-rate HTTP load testing, which is useful for open-loop benchmarking. ⁸⁵
- k6 provides explicit open-model arrival-rate executors appropriate for SLO-focused studies. ⁸⁶

Core metrics to report

Service-level metrics

- Request throughput (req/s) and, for LLMs, **tokens/s** (separately for input vs output if possible).
- End-to-end latency distribution (p50, p95, p99).
- For streaming LLM serving: time-to-first-token (TTFT) and per-token inter-arrival jitter (if available).
- Error rates (timeouts, 429/5xx, OOM failures).

System-level metrics

- GPU utilization, SM utilization proxies, and GPU memory usage. DCGM Exporter exposes GPU metrics at `/metrics` for Prometheus scraping. ⁸⁷
- GPU power/energy: NVML documents power reporting semantics, and Triton Perf Analyzer can collect server-side metrics including GPU utilization and GPU power usage by querying the Prometheus metrics endpoint. ⁸⁸
- CPU utilization and memory (to avoid “CPU-bound tokenization” masquerading as “GPU inference inefficiency”).

A practical “cost proxy” suitable for a laptop paper is **joules per 1K output tokens** or **GPU-seconds per 1K tokens**; the paper should present these as relative efficiency indicators rather than cloud-dollar cost unless you add a cloud extension. NVML uses milliwatt power readings, enabling energy calculation by integrating power over time. ⁸⁹

Metrics collection architecture

Prometheus-style `/metrics` endpoints

- Triton provides Prometheus metrics at `:8002/metrics` by default. ⁹⁰
- vLLM documents Prometheus-style metrics categorized into server-level and request-level. ⁹¹
- Ray provides guidance for monitoring Ray clusters on Kubernetes with Prometheus and Grafana. ⁹²

Service discovery and scraping on Kubernetes

Prometheus Operator documents `ServiceMonitor` (scrape a group of services) and `PodMonitor` (scrape a group of pods) CRDs; these are the typical K8s-native way to keep monitoring configuration inside your repo. ⁹³

Minimal example (ServiceMonitor pattern; adapt to your environment):

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
```

```
name: inference-metrics
spec:
  selector:
    matchLabels:
      app: triton
  endpoints:
  - port: metrics
    path: /metrics
    interval: 5s
```

(Record the exact scrape interval; it can affect overhead and granularity.)

Experiment design for reproducibility, pitfalls, and concrete deliverables

Reproducible experiment design blueprint

To be “paper-grade,” every figure should be reproducible from a tagged commit and pinned container images.

Configuration control and pinning

- Pin **container tags and preferably digests** for each serving system; Triton’s official compatibility matrix maps container tags to CUDA driver requirements and bundled library versions—record both host driver and chosen tag. ⁵²
- Keep a single “source of truth” config file per system (Helm values or YAML), committed to the repo.
- For LLMs, pin model revision identifiers and set deterministic controls: vLLM documents `--seed`; its benchmarking CLI also supports deterministic dataset ordering via `--disable-shuffle`. ⁹⁴

Experimental procedure control

- Define a standard run structure: *warm-up* → *measurement window* → *cool-down*, and discard warm-up samples.
- Repeat each condition multiple times (e.g., 5–10) and report confidence intervals (bootstrap is often appropriate for latency percentiles).
- If using Triton Perf Analyzer, note that it repeats measurements until stable values and can stabilize based on percentile via `--percentile`, which fits naturally with tail-latency-driven papers. ⁹⁵

Statistical analysis and reporting

- Report both **absolute metrics** (p99 latency, tokens/s) and **normalized metrics** (relative to a baseline stack).
- Include variance across repetitions; large variance is itself a finding on laptop GPUs (thermal throttling or background jitter).
- Use “threats to validity” section to explicitly bound generalization (single node, single GPU, laptop thermal constraints, device-plugin behavior differences).

Experiment matrix (template)

This is an explicit experiment matrix you can include in the paper and implement as a parameter grid in code. Adapt the levels to your runtime budget.

Dimension	Baseline levels (recommended)	Notes for rigor
System under test	Direct server; multi-framework server; orchestration/operator layer	Keep comparisons within the same layer when possible <code>filecite{turn0file0}</code>
Protocol	V2 inference protocol where possible; OpenAI-compatible for LLM endpoints	Reduces client-side confounds ⁹⁶
Model tier	“Smoke” small; “primary” small; “stress” medium (quantized if needed)	Publish envelopes, not only peak throughput ⁹⁷
Precision	FP16 baseline; add INT8/INT4 as second phase	vLLM + TensorRT-LLM both document quantization pathways ⁹⁸
Input size	3 bins (short/medium/long)	Must be identical across systems
Output cap (LLM)	short/medium/long generation	Separate “prefill-heavy” vs “decode-heavy” regimes ⁹⁹
Load model	Closed-loop concurrency; open-loop arrival rate; bursty ramps	k6 arrival-rate + vLLM burstiness are directly documented knobs ¹⁰⁰
Concurrency / rate	Sweep until either OOM or SLO violation	Record failure modes explicitly
Replications	≥5 per condition	Report confidence intervals
Monitoring	Same scrape interval and metric set	Otherwise monitoring overhead becomes a hidden variable

Known pitfalls and mitigations (laptop GPU + local Kubernetes)

Thermal throttling and power management

Latency spikes can align with GPU temperature/power changes; mitigate by logging GPU temperature and power continuously (NVML/nvidia-smi) and running on stable power conditions (AC). NVIDIA documents both `nvidia-smi` logging patterns and NVML power behavior. ²⁰

Driver/container tag incompatibility

Triton containers have explicit CUDA driver requirements; mitigations are to select tags using the compatibility matrix and record both tag and host driver version in the paper. ⁵²

GPU sharing confounds

If you enable time-slicing to run multiple pods, remember it multiplexes compute without memory/fault isolation; interference can inflate tail latency and create non-stationary behavior. Use a strict “no sharing” baseline and add sharing only as a separate experiment phase. ⁶⁶

If you discuss MIG, note it is supported on specific data center GPUs; consult NVIDIA's supported GPU list and clarify whether your laptop GPU supports MIG (most consumer GPUs do not). ¹⁰¹

Autoscaling that cannot increase GPU capacity

HPA replication on a single node does not create more GPU capacity unless you explicitly oversubscribe; for Standard mode, KServe documents use of HPA and notes Standard mode lacks scale-to-zero support. Explicitly state what “autoscaling” means in your experiments. ¹⁰²

Protocol mismatches and client-side artifacts

Mixing request formats (OpenAI vs custom JSON vs V2 protocol) can turn the “benchmark” into a “client compatibility test.” Prefer V2 protocol where supported, and otherwise implement a consistent OpenAI-compatible client for LLM endpoints. ¹⁰³

Concrete deliverables to include in the paper and artifact repo

Tables

- A “systems table” (like the survey table above) plus a smaller “systems under test” table that states: *layer, protocol, image tag/digest, model, quantization, key knobs*.
- An experiment matrix table (the template above) with the exact executed subset highlighted.

Baseline configurations (recommended to publish verbatim as appendices)

- GPU enablement manifest(s) (device plugin/operator) and exact container runtime configuration steps. ¹⁰⁴
- Triton: container tag + ports + metrics enablement and `:8002/metrics` availability. ¹⁰⁵
- vLLM-style server: `--seed`, `--max-model-len`, `--gpu-memory-utilization`, and whether quantization is enabled. ¹⁰⁶

Suggested figures/plots (high signal-to-effort on a laptop)

- Throughput vs p99 latency curves (Pareto fronts) at fixed workload shapes.
- p99 latency vs concurrency for each workload shape (short/long prefill/decode-heavy).
- Time series: GPU utilization, GPU memory used, GPU power, and p99 latency during step-load and burst-load runs. DCGM Exporter + Prometheus-style endpoints make this straightforward. ¹⁰⁷
- “Stability envelope” heatmap: success/failure (OOM, timeouts) across (concurrency × context length × model size).

Requested diagrams

Architecture diagram (benchmark harness and observability):

```
flowchart LR
    subgraph K8S["Local Kubernetes (single node)"]
        subgraph GPU["GPU enablement"]
            RT["Container runtime GPU config\n(nvidia-ctk + runtime)"]
            DP["Device plugin or GPU operator\nexposes nvidia.com/gpu"]
            MON["GPU telemetry (DCGM exporter)\n/metrics"]
        end
    end
```

```

subgraph SUT["Systems under test"]
  MS1["Inference server A\n(LLM or predictive)"]
  MS2["Inference server B\n(alt backend/engine)"]
  OP["Optional operator/orchestrator layer\n(CRD-managed deployments)"]
end

MET["Metrics stack\nPrometheus (+ Grafana)"]
end

LG["Load generator\n(closed-loop + open-loop)"] --> SUT
SUT --> MET
MON --> MET

```

Experiment timeline (example you can adapt to your schedule):

```

gantt
  title Laptop GPU Kubernetes Inference Benchmark Timeline
  dateFormat YYYY-MM-DD
  axisFormat %b %d

  section Environment
    GPU driver + nvidia-smi validation           :a1, 2026-03-15, 3d
    Container runtime GPU config (nvidia-ctk)    :a2, after a1, 2d
    Local Kubernetes w/ nvidia.com/gpu working  :a3, after a2, 3d

  section Deploy + verify
    Baseline inference server + metrics          :b1, after a3, 3d
    Alternative server/backend + metrics         :b2, after b1, 3d
    Optional operator/orchestrator layer         :b3, after b2, 4d

  section Benchmark suite
    Workload suite frozen + hashed              :c1, after b1, 3d
    Load profiles (open/closed/bursty)         :c2, after c1, 2d
    Monitoring dashboards + scrape config       :c3, after c1, 3d

  section Experiments
    Concurrency sweeps + Pareto curves          :d1, after c2, 5d
    VRAM envelope (max len / batch) study       :d2, after d1, 4d
    Quantization + batching sensitivity         :d3, after d2, 5d

  section Writing + artifacts
    Methods + threats to validity               :e1, after c2, 4d
    Results + discussion                       :e2, after d3, 6d
    Repro repo tag + camera-ready polish       :e3, after e2, 4d

```

1 7 70 99 Taming Throughput-Latency Tradeoff in LLM Inference with ...

https://www.usenix.org/system/files/osdi24-agrawal.pdf?utm_source=chatgpt.com

2 8 Efficient Memory Management for Large Language Model Serving with PagedAttention

https://arxiv.org/abs/2309.06180?utm_source=chatgpt.com

3 Device Plugins

https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/?utm_source=chatgpt.com

4 5 17 90 Metrics — NVIDIA Triton Inference Server

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/metrics.html?utm_source=chatgpt.com

6 91 Metrics - vLLM

https://docs.vllm.ai/en/stable/design/metrics/?utm_source=chatgpt.com

9 57 tensorrtllm_backend/docs/model_config.md at main

https://github.com/triton-inference-server/tensorrtllm_backend/blob/main/docs/model_config.md?utm_source=chatgpt.com

10 71 96 103 Open Inference Protocol (V2 Inference Protocol)

https://kserve.github.io/website/docs/concepts/architecture/data-plane/v2-protocol?utm_source=chatgpt.com

11 79 102 Autoscaling with Kubernetes HPA

https://kserve.github.io/website/docs/model-serving/predictive-inference/autoscaling/hpa-autoscaler?utm_source=chatgpt.com

12 20 nvidia-smi

https://docs.nvidia.com/deploy/nvidia-smi/?utm_source=chatgpt.com

13 14 104 Installing the NVIDIA Container Toolkit

https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html?utm_source=chatgpt.com

15 46 47 bentoml/Yatai: Model Deployment at Scale on Kubernetes ...

https://github.com/bentoml/Yatai?utm_source=chatgpt.com

16 72 NVIDIA TensorRT Documentation

https://docs.nvidia.com/deeplearning/tensorrt/latest/index.html?utm_source=chatgpt.com

18 21 Using NVIDIA GPUs with minikube

https://minikube.sigs.k8s.io/docs/tutorials/nvidia/?utm_source=chatgpt.com

19 65 NVIDIA device plugin for Kubernetes

https://github.com/NVIDIA/k8s-device-plugin?utm_source=chatgpt.com

22 Schedule GPUs

https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/?utm_source=chatgpt.com

23 Add-on: gpu - MicroK8s

https://canonical.com/microk8s/docs/addon-gpu?utm_source=chatgpt.com

24 Advanced Options / Configuration

https://docs.k3s.io/advanced?utm_source=chatgpt.com

25 26 Control Plane API

https://kserve.github.io/website/docs/reference/crd-api?utm_source=chatgpt.com

27 29 **Development Guide**

https://kserve.github.io/website/docs/developer-guide?utm_source=chatgpt.com

28 **KServe becomes a CNCF incubating project**

https://www.cncf.io/blog/2025/11/11/kserve-becomes-a-cncf-incubating-project/?utm_source=chatgpt.com

30 32 **The next generation of KFServing**

https://kserve.github.io/website/blog/kfserving-transition?utm_source=chatgpt.com

31 **v0.7.0 · kserve kserve · Discussion #1866**

https://github.com/kserve/kserve/discussions/1866?utm_source=chatgpt.com

33 35 **kserve/modelmesh: Distributed Model Serving Framework**

https://github.com/kserve/modelmesh?utm_source=chatgpt.com

34 **ModelMesh Installation Guide**

https://kserve.github.io/website/docs/admin-guide/modelmesh?utm_source=chatgpt.com

36 39 69 **Introduction**

https://www.kubeflow.org/docs/components/kserve/introduction/?utm_source=chatgpt.com

37 **Models Web App**

https://www.kubeflow.org/docs/components/kserve/webapp/?utm_source=chatgpt.com

38 **Kubeflow KServe**

https://www.kubeflow.org/docs/components/kserve/?utm_source=chatgpt.com

40 41 44 **Architecture | v2.9 | Seldon Core 2**

https://docs.seldon.ai/seldon-core-2/v2.9/about/architecture?utm_source=chatgpt.com

42 **Seldon Core v2**

https://deploy.seldon.io/en/v2.2/contents/getting-started/production-installation/core-v2.html?utm_source=chatgpt.com

43 **README.md - SeldonIO/seldon-core**

https://github.com/SeldonIO/seldon-core/blob/v2/README.md?utm_source=chatgpt.com

45 **BentoDeployment CRD - Yatai**

https://docs.yatai.io/en/latest/concepts/bentodeployment_crd.html?utm_source=chatgpt.com

48 **Work with GPUs**

https://docs.bentoml.com/en/latest/build-with-bentoml/gpu-inference.html?utm_source=chatgpt.com

49 **TorchServe — PyTorch/Serve master documentation**

https://docs.pytorch.org/serve/?utm_source=chatgpt.com

50 105 **Running Triton**

https://docs.nvidia.com/deeplearning/triton-inference-server/archives/triton_inference_server_1140/user-guide/docs/run.html?utm_source=chatgpt.com

51 83 95 **Performance Analyzer — NVIDIA Triton Inference Server**

https://docs.nvidia.com/deeplearning/triton-inference-server/archives/triton-inference-server-2310/user-guide/docs/user_guide/perf_analyzer.html?utm_source=chatgpt.com

52 53 54 **Release Compatibility Matrix — NVIDIA Triton Inference ...**

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/introduction/compatibility.html?utm_source=chatgpt.com

55 56 58 **TensorRT-LLM Backend — NVIDIA Triton Inference Server**

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/tensorrtllm_backend/README.html?utm_source=chatgpt.com

59 **Model Configuration — NVIDIA Triton Inference Server**

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/model_configuration.html?utm_source=chatgpt.com

60 **Deploy on Kubernetes - Ray Serve**

https://docs.ray.io/en/latest/serve/production-guide/kubernetes.html?utm_source=chatgpt.com

61 **vLLM compatibility — Ray 2.54.0**

https://docs.ray.io/en/latest/serve/llm/user-guides/vllm-compatibility.html?utm_source=chatgpt.com

62 63 **Production Guide — Ray 2.54.0 - Ray Docs**

https://docs.ray.io/en/latest/serve/production-guide/index.html?utm_source=chatgpt.com

64 **kuberay-operator**

https://ray-project.github.io/kuberay/deploy/helm/?utm_source=chatgpt.com

66 67 **Time-Slicing GPUs in Kubernetes — NVIDIA GPU Operator**

https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-sharing.html?utm_source=chatgpt.com

68 **Installing the NVIDIA GPU Operator**

https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/getting-started.html?utm_source=chatgpt.com

73 94 97 106 **Engine Arguments - vLLM**

https://docs.vllm.ai/en/stable/configuration/engine_args/?utm_source=chatgpt.com

74 **vLLM Backend — NVIDIA Triton Inference Server**

https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/vllm_backend/README.html?utm_source=chatgpt.com

75 **BitsAndBytes - vLLM**

https://docs.vllm.ai/en/stable/features/quantization/bnb/?utm_source=chatgpt.com

76 **Numerical Precision — TensorRT-LLM**

https://nvidia.github.io/TensorRT-LLM/reference/precision.html?utm_source=chatgpt.com

77 86 100 **Constant arrival rate | Grafana k6 documentation**

https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/constant-arrival-rate/?utm_source=chatgpt.com

78 82 **Benchmark CLI - vLLM**

https://docs.vllm.ai/en/latest/benchmarking/cli/?utm_source=chatgpt.com

80 **Configuring scale to zero**

https://knative.dev/docs/serving/autoscaling/scale-to-zero/?utm_source=chatgpt.com

81 **Prometheus Adapter for Kubernetes Metrics APIs**

https://github.com/kubernetes-sigs/prometheus-adapter?utm_source=chatgpt.com

84 **What is Locust?**

https://docs.locust.io/en/stable/what-is-locust.html?utm_source=chatgpt.com

85 **tseart/vegeta: HTTP load testing tool and library. It's over ...**

https://github.com/tseart/vegeta?utm_source=chatgpt.com

87 107 DCGM Exporter — NVIDIA GPU Telemetry 1.0.0 ...

https://docs.nvidia.com/datacenter/cloud-native/gpu-telemetry/latest/dcgm-exporter.html?utm_source=chatgpt.com

88 89 nvmlDeviceGetClockInfo

https://docs.nvidia.com/deploy/nvml-api/group__nvmlDeviceQueries.html?utm_source=chatgpt.com

92 Using Prometheus and Grafana — Ray 2.54.0

https://docs.ray.io/en/latest/cluster/kubernetes/k8s-ecosystem/prometheus-grafana.html?utm_source=chatgpt.com

93 API reference - Prometheus Operator

https://prometheus-operator.dev/docs/api-reference/api/?utm_source=chatgpt.com

98 Quantization - vLLM

https://docs.vllm.ai/en/v0.10.2/features/quantization/index.html?utm_source=chatgpt.com

101 Supported GPUs — NVIDIA Multi-Instance GPU User Guide

https://docs.nvidia.com/datacenter/tesla/mig-user-guide/supported-gpus.html?utm_source=chatgpt.com